

Volatility & Position Risk Calculator

Design Document: Mathematics, Metric Redefinition, and Build Plan

This document explains, in full mathematical and engineering detail, (1) exactly what the current PositionCalculator notebook computes today, (2) how its Greek definitions map onto the desk's target conventions and what specifically needs to change, and (3) the design for every requested extension — breakevens, payoff diagrams, multi-leg structures, tenor/stress risk views, a strike map, visuals, an application layer, and CSV I/O — so a build decision can be made from a single reference. No code has been changed; this is the design and rationale to review before implementation.

Prepared for review | Source: PositionCalculator.ipynb

Contents

1. Current System — Architecture Overview
2. Current System — Mathematics of Every Computed Quantity
3. Target Metric Definitions — Gap Analysis
4. New Analytics: Breakevens
5. New Analytics: Payoff Diagrams
6. New Analytics: Multi-Leg Structures
7. Portfolio Risk Views (Tenor, Stress Ladders, Strike Map)
8. Visualization Plan
9. Application Architecture & Delivery (Modules, UI, CSV I/O)
10. Recommended Build Roadmap

1. Current System — Architecture Overview

The notebook implements a small options/futures risk engine on top of QuantLib. It is organized as a single evaluation pipeline: **instruments** (what you hold) are priced against a **market** (current conditions) by a **pricer** (QuantLib), and a **Greek engine** differentiates that pricing numerically to produce risk sensitivities. Everything downstream — ladders, surfaces, plots — is built by repeatedly re-running this same pipeline under shocked inputs.

1.1 Core objects

Class	Role	Key attributes
Market	A snapshot of live conditions.	futures_price (F), interest_rate (r)
Future	Linear futures position.	quantity (q)
AmericanOption	A single option leg. Each leg carries its own volatility — there is no shared market-wide vol.	quantity, strike (K), expiry_days (τ), option_type, volatility (σ)
AmericanOptionPricer	Thin wrapper around QuantLib: builds a Black-Scholes process + American exercise + Barone-Adesi-Whaley engine, on every call.	F, K, r, σ , τ , option_type
Portfolio	Ordered collection of positions. <i>value(market)</i> sums <i>price(market)</i> across all positions — this polymorphism is what lets every downstream function stay instrument-agnostic.	positions: list
GreekEngine	Stateless. Every method takes (portfolio, market) and returns a numeric sensitivity by finite-differencing <code>Portfolio.value()</code> .	static methods only

1.2 Pricing engine mechanics (QuantLib)

Each call to `AmericanOptionPricer.price()` performs, from scratch:

- Sets the global QuantLib evaluation date to *today* (mutated on every call — safe here since it is always reset to the same value, but not thread-safe if ever parallelized).
- Builds an `AmericanExercise` from *today* to *today* + *expiry_days* — expiry is a day-count offset, not a fixed calendar date. This is why time-decay is simulated by decrementing *expiry_days* rather than moving the evaluation date.
- Constructs a `BlackScholesProcess` from a flat spot quote (F), a flat `FlatForward` rate curve (r), and a flat `BlackConstantVol` curve (σ) — i.e. no term structure and no smile; each leg's own σ stands in for a smile point.
- Prices with `BaroneAdesiWhaleyApproximationEngine` — a fast, closed-form *approximation* for American exercise (not a binomial tree or PDE grid). Good speed/accuracy trade-off for a risk engine that reprices constantly, with known accuracy degradation very close to expiry, at the money.

Note: F is passed into a Black-Scholes (spot) process rather than a Black-76 (forward) process. For undiscounted futures this is a common workaround, but it means the interest-rate curve is doing double duty — both discounting and drift — with no separate carry/dividend curve.

1.3 Why every Greek is a repricing loop, not a formula

QuantLib does not expose an analytic Greek for the Barone-Adesi-Whaley engine here, so `GreekEngine` computes **all** sensitivities by finite differences on `Portfolio.value()`: bump an input, deep-copy the portfolio or market, reprice the whole book, and difference. This is simple and robust but means a single `GreekEngine.report()` call triggers dozens of full QuantLib repricings for even a 3-leg book — worth keeping in mind as more legs, tenors, and stress axes are added (Sections 7–9).

2. Current System — Mathematics of Every Computed Quantity

Let $V(F, \{\sigma_i\}, r)$ denote total portfolio value: $V = q_{\text{future}} \cdot F + \sum_i q_i \cdot P_i(F, \sigma_i, r, \tau_i)$, where P_i is the American price of leg i . Bump sizes used throughout: $\text{SPOT_BUMP} = 0.001$ (0.1% of F), $\text{VOL_POINT} = 0.01$ (1 percentage point of vol), $\text{BASIS_POINT} = 0.0001$ (1bp of rate).

2.1 Delta

```
ds = F × SPOT_BUMP
Delta = [ V(F+ds) - V(F-ds) ] / (2·ds) ≈ ∂V/∂F
```

A central-difference estimate of $\partial V / \partial F$ (second-order accurate, error $\sim O(ds^2)$). Since $\text{Future.price} = q \cdot F$ contributes exactly q to the derivative, and each option's $\partial P_i / \partial F$ is its own Black-Scholes-style delta (dimensionless, roughly 0 to ± 1), the sum collapses to **Delta** = $q_{\text{future}} + \sum_i q_i \cdot \delta_i$ — a futures-equivalent *contract count*.

2.2 Gamma

```
Gamma = [ V(F+ds) - 2·V(F) + V(F-ds) ] / ds² ≈ ∂²V/∂F² = ∂Delta/∂F
```

The standard 3-point central second-derivative stencil. Because the derivative is taken directly against F (in whatever units F is quoted — cents/bu for a grain future), this is already **change in delta-contracts per 1-unit move in F** .

2.3 Vega

```
dv = VOL_POINT = 0.01 (every leg's σi shifted ±dv, in parallel)
raw = [ V(σ+dv) - V(σ-dv) ] / (2·dv) ≈ ∂V/∂σ
Vega = raw × VOL_POINT
```

The raw central difference gives $\partial V / \partial \sigma$ per unit of *decimal* vol (i.e. per 100 percentage points); multiplying by VOL_POINT rescales it down to **\$ change in V for a 1-percentage-point parallel shift** in every leg's implied vol — the standard trading-desk vega convention.

2.4 Theta

```
Theta = V(portfolio one day closer to expiry) - V(portfolio today)
```

Not a continuous derivative — a literal one-step simulation. Every leg's `expiry_days` is decremented by 1 (floored at 1) via `shift_time`, holding F , σ , and r fixed, and the book is repriced. This directly equals **\$ P&L from one day of decay**. Sign convention: a long option (losing time value) shows up negative, matching standard usage.

2.5 Rho

```
dr = BASIS_POINT = 0.0001 (1bp)
raw = [ V(r+dr) - V(r-dr) ] / (2·dr) ≈ ∂V/∂r
Rho = raw × BASIS_POINT
```

Same pattern as Vega, but rescaled to **\$ change in V per 1 basis point** (0.01%) move in the risk-free rate.

2.6 Second-order / cross Greeks

Greek	Formula	Meaning
Volga (vomma)	$[V(\sigma+dv) - 2V(\sigma) + V(\sigma-dv)] / dv^2 \times \text{VOL_POINT}$	$\partial^2 V / \partial \sigma^2 = \partial \text{Vega} / \partial \sigma$ — convexity of vega to vol.
Vanna (via vega)	$[\text{Vega}(F+ds) - \text{Vega}(F-ds)] / (2ds)$	$\partial^2 V / \partial F \partial \sigma$ computed as a spot-difference of vega.
Vanna (via delta)	$[\text{Delta}(\sigma+dv) - \text{Delta}(\sigma-dv)] / (2dv)$	The same cross-partial, computed as a vol-difference of delta instead. Both are reported; by symmetry of mixed partials they should closely agree — small discrepancies are pure finite-difference error and act as a built-in consistency check, not a bug.

These 'nested' finite differences (a difference of a difference) are the most expensive calls in the engine — each one reprices the full book roughly 4 times.

3. Target Metric Definitions — Gap Analysis

The desk's target definitions were compared line-by-line against what the code already computes. Four of five already match exactly; one (Rho) has a real unit mismatch.

Greek	Target definition	Currently computes	Gap
Delta	Contracts	$q_{\text{future}} + \sum q_i \cdot \delta_i$ (contracts)	None. Already in contracts.
Gamma	Δ delta-contracts per 1¢/bu move	$\partial^2 V / \partial F^2$, i.e. Δ delta per 1 unit of F	None , provided F is quoted in cents/bu.
Vega	\$k per 1% vol move	\$ per 1 vol point (1 percentage point)	Display scale only (\$→\$k); no formula change.
Theta	\$k per 1-day decay	\$ for exactly 1 day of decay	Display scale only (\$→\$k); no formula change.
Rho	\$k per 1% rates move (100bp)	\$ per 1bp	Real change: rescale factor 0.0001 → 0.01 (i.e. multiply current output by 100), plus \$→\$k.

3.1 Open question before implementing the \$k conversions

Nowhere in the current code is there a **contract multiplier** (e.g. \$/cent × bushels-per-contract for a grain future). V() today is quantity × raw price units, not verified dollars. Before wiring up \$k display, confirm whether F/strike are already effectively \$-per-contract (so \$k is a one-line divide-by-1000), or whether a multiplier constant needs to be threaded through Market or the instrument classes first.

4. New Analytics: Gamma / Theta Breakevens

Definition: the spot move ΔF large enough that the gamma P&L from that move offsets the theta decay paid over the period. Derived from a second-order Taylor expansion of P&L around current spot:

$$\Delta V \approx \text{Delta} \cdot \Delta F + \frac{1}{2} \cdot \text{Gamma} \cdot \Delta F^2 + \text{Theta}$$

Setting $\Delta V = 0$ and solving the quadratic for ΔF gives two breakeven points (upside and downside):

$$\Delta F = [-\text{Delta} \pm \sqrt{\text{Delta}^2 - 2 \cdot \text{Gamma} \cdot \text{Theta}}] / \text{Gamma}$$

Breakeven *price levels* are then $F \pm \Delta F$. This is cheap: Delta/Gamma/Theta already come out of GreekEngine, so a new `breakevens(portfolio, market)` function is just this quadratic applied to numbers already being computed.

4.1 A more accurate alternative

The Taylor/quadratic approach degrades for large moves or strongly convex payoffs (e.g. butterflies, Section 6). Since a real pricer is already available, a numerically exact alternative is preferable: use `shift_time(portfolio, 1)` to get tomorrow's book, then root-find (e.g. `scipy.optimize.brentq`) on

$$f(\Delta F) = V(\text{shifted_portfolio}, F + \Delta F) - V(\text{portfolio}, F)$$

for the ΔF where next-day P&L crosses zero — no Taylor error, at the cost of a handful of repricings via bisection instead of one closed-form quadratic. **Recommendation:** keep the quadratic as an instant/cheap number in the summary table, but use the numerical root-find as the primary, more trustworthy breakeven once kinked structures (butterflies, 3-ways) are in the book.

5. New Analytics: Payoff Diagrams

5.1 Payoff at expiry

At expiry, every option collapses to pure intrinsic value — no vol, no rate, no time value:

```
Call: max(F - K, 0) Put: max(K - F, 0)
```

Rather than pushing `expiry_days` to 0 and asking QuantLib to price a 0-day American option (an edge case worth avoiding), intrinsic value should be computed directly in Python — exact, and fast across however many spot points are needed for a smooth curve. The Future leg contributes $q \cdot F$ as always. For a **P&L** diagram (not just terminal value), subtract today's portfolio value as cost basis — the same ' $\text{PnL} = \text{value} - \text{current_value}$ ' pattern already used by `spot_ladder` / `vol_ladder`. This convention already nets out correctly for the future leg too: P&L becomes $q \cdot (F_{\text{final}} - F_{\text{today}})$ with no new field needed on Future.

5.2 Increments to expiry

Reuses `shift_time` (already in the codebase) combined with a spot sweep. For each time step in a list such as `[0, 7, 14, ..., expiry_days]`, call `shift_time(portfolio, step)` to get the book at that horizon, then sweep spot across a range computing value at each point — the same inner loop as `spot_ladder`, but lighter (value only, no Greeks needed at every point). Plot one line per time step overlaid on shared axes — the familiar 'T+0, T+7, ..., T+expiry' payoff diagram used across most options-analysis tools.

6. New Analytics: Multi-Leg Structures

Design principle: do not add seven new option-like classes. Keep `AmericanOption` / `Future` exactly as they are, and add a thin **composition layer** on top.

6.1 Architecture

- **Common interface.** Replace the `isinstance(pos, AmericanOption)` checks in `shift_portfolio_vols` / `shift_time` with polymorphic methods — `shift_vol(dv)`, `shift_time(days)` — implemented per class (`Future`: no-op; `AmericanOption`: shift itself). Small refactor, but it's what lets everything below work without touching `GreekEngine` again.
- **Structure composite.** A class holding legs: `list[...]` that implements the same interface: `price(market) = sum(leg.price(market) for leg in legs)`, and `shift_vol/shift_time` delegate to each leg. Since it quacks like any other instrument, `Portfolio.value()` and every `Greek/scenario` function work on it with **zero** further changes — plus a readable `__repr__` such as `'Straddle(strike=530, qty=10)'` instead of two anonymous legs.
- **Factory functions** (not classes) — each just assembles correctly-signed `AmericanOptions` into a `Structure`. No new pricing math anywhere; purely construction convenience.

6.2 Per-structure construction rules

Structure	Legs	Construction
Call / Put spread	2	Long 1× K1, short 1× K2, same type & expiry, opposite sign, same size.
Straddle	2	1 call + 1 put, same strike & expiry, same sign, same size.
Strangle	2	Same as straddle but call strike > put strike (both OTM).
Butterfly	4	Long 1× K1, short 2× K2, long 1× K3 (same type). Ratio defaults to 1:2:1, overridable for broken-wing variants.
Collar	2	Long put at K1 + short call at K2 (K1 < K2), opposite types/direction, independently sized/vol'd legs.
3-way	3	Deliberately not a fixed template — desk conventions vary (collar + extra short leg, strangle + directional option, etc.). Built as a generic n-leg builder parametrized per house convention rather than one hard-coded shape.
Calendar spread	2	Same strike & type, different expiry_days. Already fully supported by the current data model with zero change — each <code>AmericanOption</code> already carries its own independent expiry_days, and <code>shift_time</code> already decrements each leg's expiry independently. Only the convenience constructor is new.

Net effect: `AmericanOption` / `Future` / `Portfolio` / `GreekEngine` stay almost untouched (just the `isinstance`→polymorphism refactor). The 7 structures become ~10–20 line factory functions in a new `structures.py` — and each factory is independently unit-testable by checking its aggregate Greeks match manually summing its individual legs.

7. Portfolio Risk Views: Tenor, Stress Ladders, Strike Map

7.1 Portfolio Greeks ATM and across tenor

Group `portfolio.positions` by tenor (expiry bucket), build a temporary sub-Portfolio per bucket, and run the existing `GreekEngine.report()` on each — plus once more on the full book for a Total row. No Greek math changes; this is a partition applied before calling code that already exists. Output: a DataFrame with rows = tenor buckets + Total, columns = the Greeks.

Wrinkle: a calendar spread (Section 6) spans two tenors by design. Tenor bucketing needs to flatten Structures down to their underlying single legs first (each leg still knows its own `expiry_days`) — a calendar spread then correctly shows up split across its two tenor rows.

7.2 Stress ladders — stress one variable, hold others constant

A generalization of the existing `spot_ladder` / `vol_ladder`, with two changes: it reports **one specific Greek's value** at each stress level (not everything at once), and price stress needs **fixed absolute cent steps** rather than the existing percentage-based steps.

```
stress_table(portfolio, market, greek, axis, levels)
axis = 'price' | 'time' | 'vol'
```

Each axis maps directly onto machinery that already exists in the notebook:

Axis	Shock applied	Reused from
price	±5 / 10 / 15 / 20 cents added directly to <code>market.futures_price</code>	New: an absolute-bump variant of the existing spot shock (current <code>spot_ladder</code> is percentage-only).
time	<code>shift_time(portfolio, level)</code> , level ∈ {5,10,15,20} days, forward only	Reused as-is.
vol	<code>shift_portfolio_vols(portfolio, level/100)</code> , level ∈ {1,2,3,4} vol points, ±	Reused as-is, just swept over the specific required levels instead of a linspace range.

The required 8 combinations become different call arguments into one generic function:

Greek	vs. price	vs. time	vs. vol
Delta	✓	✓	✓
Gamma	✓	✓	✓
Vega	✓	✓	—
Theta	✓	✓	—

Vega-vs-vol and Theta-vs-vol are intentionally absent from the requirements — those are volga and a veta-type cross-Greek respectively (different things), not requested here.

7.3 Portfolio strike map by expiry

A pivot table: rows = strike, columns = expiry/tenor, cell = aggregated quantity (or a Greek, e.g. net delta) for legs sharing that strike/expiry. Since every leg already exposes strike, `expiry_days`, `option_type`, quantity as plain attributes, this is a direct pandas job:

```
flatten Structures to legs → build flat DataFrame of leg attributes →  
df.pivot_table(index='strike', columns='expiry_days', values='quantity',  
aggfunc='sum')
```

For a Greek-weighted version, compute each leg's own delta in isolation (same finite-difference math as `GreekEngine.delta`, scoped to a single position) before pivoting.

8. Visualization Plan

View	Chart type	Rationale
Strike map (7.3)	Heatmap (seaborn, same pattern as plot_surface)	Strike rows × expiry columns, colored by net quantity or net delta — a genuinely 2D relationship.
Stress ladders (7.2)	Line chart(s), one panel per Greek×axis combination (up to 8), or grouped by Greek with price/time/vol as separate lines	These are 1D sweeps of one Greek against one stressed axis — a heatmap would obscure magnitude/direction that a line makes obvious.
Spot × Vol surface (existing)	Heatmap (already implemented via plot_surface)	Kept as-is — genuinely 2D (spot move × vol shift).
Payoff diagrams (5)	Overlaid line chart, one line per time-to-expiry increment	Standard 'T+0 ... T+expiry' options payoff convention.

9. Application Architecture & Delivery

9.1 Module split

```
vol_calculator/  
  market.py      Market  
  instruments.py  Portfolio, Future, AmericanOption, Structure (composite)  
  structures.py   call_spread, put_spread, straddle, strangle,  
                  butterfly, collar, three_way, calendar_spread  
  pricing.py      AmericanOptionPricer  
  greeks.py       GreekEngine, shift_portfolio_vols, shift_time  
  scenarios.py    spot_ladder, vol_ladder, spot_vol_surface,  
                  stress_table, breakevens, payoff_diagram, strike_map  
  plotting.py     plot_surface + the new line-chart / heatmap views (Sec. 8)  
  io.py           portfolio_to_csv / portfolio_from_csv, export_report  
  
notebooks/  
  PositionCalculator.ipynb  thin: imports vol_calculator, keeps the demo /  
                           worked-example cells for the notebook-only user
```

This is the same split proposed earlier in review, now extended with the two new modules (structures.py, io.py) that this round of requirements introduces. Everything in Sections 4–8 is additive to this layout — no existing module's public surface needs to change, only grow.

9.2 User interface — options and recommendation

Option	Effort	Trade-off
ipywidgets in the existing notebook	Lowest	Stays inside the environment the notebook-only user already has; sliders/dropdowns for stress axes, live-updating tables/plots in-cell. No new deployment story, but limited as a standalone 'desk tool'.
Streamlit app	Moderate	A real point-and-click tool (portfolio builder, live Greeks, stress ladders, strike map, heatmaps) with tabs/filters. Pure Python, reuses the existing pandas/matplotlib output almost as-is. Recommended if the goal is something that looks and behaves like a desk risk tool rather than a notebook.
Dash	Moderate–High	Similar territory to Streamlit with more layout control, at the cost of materially more boilerplate.
Desktop app (Tkinter/PyQt)	High	Meaningfully more effort for no real benefit over Streamlit here — not recommended.

Whichever is chosen, the architectural rule is the same one from the module split: the UI is a thin layer that only ever calls into vol_calculator functions — it never reimplements pricing or Greek logic. That keeps the notebook and the UI in sync automatically, since both import the same package as their single source of truth.

9.3 CSV import / export

Portfolio I/O: one flat CSV, one row per leg — instrument_type, quantity, strike, expiry_days, option_type, volatility (blank strike/expiry/vol for Future rows), plus a structure_id / structure_type column so a reload can regroup legs back into their Structure (call spread, straddle, etc.) instead of a bag of anonymous legs. portfolio_to_csv / portfolio_from_csv live in io.py.

Report I/O: nearly free — spot_ladder, vol_ladder, spot_vol_surface, the new stress_table, and the strike map are all pandas DataFrames already (or would be), so it's .to_csv() plus a consistent output-folder convention.

10. Recommended Build Roadmap

A suggested phase ordering, sequenced so each phase only depends on work already completed in an earlier phase — nothing here has been built yet; this is the order to decide against.

Phase	Scope	Depends on
0	Split the notebook into the vol_calculator package (Sec. 9.1); notebook becomes a thin consumer. No behavior change.	—
1	Fix Rho scale (Sec. 3); wire $\$ \rightarrow \k display once the contract-multiplier question (3.1) is answered.	Phase 0
2	Instrument polymorphism refactor + Structure composite + the 7 structure factories (Sec. 6).	Phase 0
3	Breakevens (Sec. 4) and payoff diagrams (Sec. 5) — both are pure additions on top of existing Greeks/shift helpers.	Phase 0 (Phase 3's numerical breakeven benefits from Phase 2's structures being in place to test against kinked payoffs)
4	Tenor-bucketed Greeks, stress ladders, strike map (Sec. 7).	Phase 2 (tenor bucketing needs Structures to be flatten-able)
5	Visualization layer for the new views (Sec. 8).	Phase 4
6	CSV I/O (Sec. 9.3).	Phase 2 (schema needs structure_id)
7	UI layer — Streamlit recommended (Sec. 9.2).	Phases 2, 4, 5, 6 (UI is a thin caller over all of the above)

End of document. This covers every item discussed in review: the current engine's mathematics, the metric gap analysis (Sections 1–3), every new analytics feature requested (Sections 4–8), and the application/delivery plan with a phase-ordered roadmap (Sections 9–10). No implementation has started — this is the reference to read before deciding what to build first.